

Kounouz Alafiya

كنوز العافية

Guide de déploiement — Kounouz Alafiya sur un VPS Hostinger

Mettre la plateforme en ligne sur un serveur privé virtuel (VPS) Hostinger, avec un nom de domaine, le HTTPS, des sauvegardes automatiques et une procédure de mise à jour sûre.

Plateforme de confiance, vérification et traçabilité du miel

Phase 1 · 20/09/2026

Sommaire

Ce guide suppose que vous partez de zéro : un compte Hostinger, un nom de domaine et le code de l'application sur votre ordinateur. Comptez environ deux heures pour une première installation. Chaque commande est à copier telle quelle ; les valeurs à remplacer sont écrites **EN_MAJUSCULES** .

1. L'application est-elle prête ?
2. Ce qu'il vous faut
3. Architecture de production
4. Préparer le code sur votre ordinateur
5. Commander et configurer le VPS Hostinger
6. Première connexion et sécurisation du serveur
7. Faire pointer le nom de domaine
8. Installer Docker
9. Installer l'application
10. Activer le HTTPS avec Caddy
11. Créer le premier administrateur
12. Vérifications après déploiement
13. Sauvegardes automatiques et restauration
14. Mettre à jour l'application
15. Surveillance et maintenance
16. Avant d'ouvrir au public
17. Dépannage

1. L'application est-elle prête ?

Techniquement, oui, après les corrections apportées le 20/09/2026 (section 1.1). **Pour le public, pas encore** : il reste des éléments de contenu et d'organisation à compléter (section 16).

Ces corrections ont été validées par une **répétition complète du déploiement** sur une pile Docker neuve : les 15 migrations s'appliquent sur une base vide et correspondent exactement au schéma, le compte administrateur se crée, la connexion passe par nginx, un envoi de 5 Mo est accepté, l'IP réelle du visiteur est enregistrée (une IP forgée par le visiteur est ignorée), les documents survivent à la recreation des conteneurs, et la sauvegarde base + fichiers est restaurable.

1.1 Ce qui a été vérifié et corrigé

Point	État
Images Docker (API NestJS, site React servi par nginx, PostgreSQL 16)	Prêt : constructions en plusieurs étapes, utilisateur non-root, sondes de santé
Migrations de base de données	Prêt : appliquées automatiquement au démarrage (<code>prisma migrate deploy</code> , jamais destructif)
Secrets JWT	Prêt : l'API refuse de démarrer sans eux
Limite de débit et IP des visiteurs derrière le proxy	Corrigé : sans <code>TRUST_PROXY</code> , tous les visiteurs partageaient une seule limite de 120 requêtes par minute et le journal des scans n'enregistrait que l'IP du proxy
Envoi de fichiers de plus de 1 Mo	Corrigé : nginx refusait tout envoi de plus de 1 Mo (documents producteur, photos, bulletins de laboratoire)
Documents d'identité des producteurs (CIN, attestations)	Corrigé : ils étaient stockés hors volume Docker et perdus à chaque mise à jour
Base de données et API exposées sur Internet	Corrigé : elles n'écoutent plus que sur 127.0.0.1 (Docker contourne le pare-feu UFW)
En-têtes de sécurité HTTP	Ajoutés (nosniff, anti-iframe, politique de référent, permissions)
Premier compte administrateur	Ajouté : script <code>create-admin.mjs</code> , sans charger les données de démonstration
Premier démarrage sur une base vide	Corrigé : l'initialisation de PostgreSQL pouvait dépasser le délai de la sonde de santé et empêcher l'API de démarrer
HTTPS	À installer : Caddy, section 10
Sauvegardes	À installer : script et tâche planifiée, section 13

1.2 Ce qui bloque encore la mise en ligne

1. **Le code n'est pas versionné.** Plus de 230 fichiers sont modifiés ou non suivis par Git, dont **toutes les migrations récentes** et **les traductions** (`frontend/src/i18n/`). Un clonage du dépôt actuel donnerait une application incomplète. Section 4.
2. **Aucun dépôt distant** (GitHub, GitLab...) n'est configuré. Section 4.
3. **Contenu provisoire** visible par le public : numéro de téléphone, codes QR de démonstration, liens légaux, avis clients. Section 16.

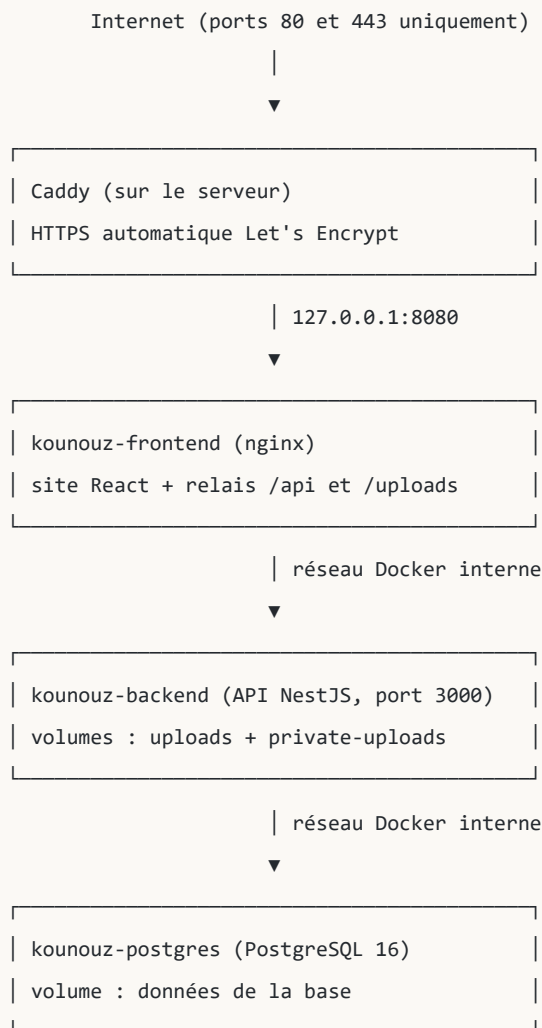
2. Ce qu'il vous faut

Élément	Recommandation
VPS Hostinger	KVM 2 (2 vCPU, 8 Go de RAM, 100 Go NVMe) : confortable pour la base, l'API, le site et la construction des images. KVM 1 (4 Go de RAM) suffit pour démarrer, avec un fichier d'échange (section 6.6)
Système	Ubuntu 24.04 LTS
Nom de domaine	Par exemple <code>kounouzalafiya.tn</code> ou <code>.com</code> , chez Hostinger ou un autre registraire
Dépôt Git privé	GitHub ou GitLab (gratuit)
Sur votre ordinateur	Git, un terminal (PowerShell sous Windows) et un client SSH (inclus dans Windows 10/11)

Dans ce guide, remplacez :

Valeur	Par
<code>VOTRE_DOMAINE</code>	votre domaine, sans <code>https://</code> ni <code>www</code> (ex. <code>kounouzalafiya.tn</code>)
<code>IP_DU_VPS</code>	l'adresse IPv4 du VPS (visible dans hPanel)
<code>VOTRE_EMAIL</code>	votre adresse e-mail d'administrateur

3. Architecture de production



- Seul Caddy est joignable depuis Internet. La base (5433) et l'API (3000) n'écoutent que sur `127.0.0.1` : accessibles depuis le serveur (ou par tunnel SSH), jamais depuis Internet.
- Le navigateur ne parle qu'à une seule adresse (`https://VOTRE_DOMAINE`) : nginx relaie `/api` vers l'API, donc pas de problème de CORS.
- Les données qui doivent survivre aux mises à jour vivent dans trois **volumes Docker** : la base, les fichiers téléversés (`uploads`) et les documents d'identité des producteurs (`private-uploads`). Ce sont eux qu'il faut sauvegarder.

4. Préparer le code sur votre ordinateur

4.1 Tout enregistrer dans Git

Dans le dossier du projet, sur votre ordinateur :

```
git status
```

Vérifiez que `.env` n'apparaît **pas** dans la liste (il contient des secrets ; il est exclu par `.gitignore`). Puis enregistrez tout :

```
git add -A
```

```
git commit -m "Version prête pour le déploiement"
```

```
git status
```

La dernière commande doit afficher « nothing to commit, working tree clean ».

4.2 Créer un dépôt privé et y envoyer le code

1. Sur <https://github.com>, créez un dépôt **privé** (bouton « New repository »), par exemple `kounouz-alafiya`, sans README.
2. Reliez votre projet à ce dépôt et envoyez le code (remplacez `VOTRE_COMPTE`) :

```
git remote add origin https://github.com/VOTRE_COMPTE/kounouz-alafiya.git
```

```
git push -u origin master
```

Ne mettez jamais le fichier `.env` de production dans Git. Il sera créé directement sur le serveur.

5. Commander et configurer le VPS Hostinger

1. Sur hostinger.com, commandez un **VPS KVM 2** (ou KVM 1).
2. Choisissez un **centre de données en Europe**, le plus proche de la Tunisie (par exemple en France) : moins de latence pour vos visiteurs.
3. Système : **Ubuntu 24.04** (modèle « OS simple »). Nous installerons Docker nous-mêmes à la section 8.
4. Définissez un **mot de passe root** long et unique, et gardez-le dans un gestionnaire de mots de passe.
5. Si hPanel le propose, ajoutez dès maintenant votre **clé SSH** (section 6.2 pour la créer).
6. Une fois le VPS prêt, notez son **adresse IPv4** dans hPanel → VPS → Aperçu.

Pare-feu Hostinger (hPanel)

hPanel → VPS → **Sécurité** → **Pare-feu** : créez une configuration avec ces règles « Accepter », puis activez-la :

Protocole	Port	Usage
TCP	22	SSH
TCP	80	HTTP (certificat Let's Encrypt, redirection vers HTTPS)
TCP	443	HTTPS

Ce pare-feu, géré par Hostinger en amont du serveur, s'ajoute à UFW (section 6.4). Il est aussi efficace contre les ports publiés par Docker, que UFW ne filtre pas.

Sauvegardes Hostinger

hPanel → VPS → **Sauvegardes et surveillance** : les sauvegardes hebdomadaires automatiques couvrent tout le serveur. Prenez aussi un **instantané (snapshot)** manuel avant chaque mise à jour importante (section 14). Ces sauvegardes ne remplacent pas celles de la section 13, qui sont quotidiennes et restaurables séparément.

6. Première connexion et sécurisation du serveur

6.1 Se connecter en root

Depuis PowerShell (Windows) ou un terminal (Mac, Linux) :

```
ssh root@IP_DU_VPS
```

Acceptez l'empreinte (**yes**) puis saisissez le mot de passe root. Mettez le système à jour :

```
apt update && apt upgrade -y
```

```
timedatectl set-timezone Africa/Tunis
```

6.2 Créer une clé SSH (sur votre ordinateur)

Si vous n'en avez pas encore, dans un **nouveau** terminal sur votre ordinateur :

```
ssh-keygen -t ed25519 -C "VOTRE_EMAIL"
```

Appuyez sur Entrée pour l'emplacement par défaut et choisissez une phrase de passe. Affichez la clé publique à copier :

```
cat ~/.ssh/id_ed25519.pub
```

6.3 Créer un utilisateur d'administration

Sur le serveur (toujours connecté en root) :

```
adduser deploy
```

```
usermod -aG sudo deploy
```

```
mkdir -p /home/deploy/.ssh && nano /home/deploy/.ssh/authorized_keys
```

Collez la clé publique (la ligne `ssh-ed25519 ...`), enregistrez (Ctrl+O, Entrée) et quittez (Ctrl+X). Puis :

```
chown -R deploy:deploy /home/deploy/.ssh && chmod 700 /home/deploy/.ssh && chmod 600  
/home/deploy/.ssh/authorized_keys
```

Sans fermer la session root, ouvrez un nouveau terminal et vérifiez que la connexion par clé fonctionne :

```
ssh deploy@IP_DU_VPS
```

6.4 Pare-feu UFW

Sur le serveur :

```
sudo ufw allow OpenSSH && sudo ufw allow 80/tcp && sudo ufw allow 443/tcp
```

```
sudo ufw enable
```

```
sudo ufw status
```

6.5 Interdire la connexion root et par mot de passe

Seulement après avoir vérifié la connexion `deploy` par clé :

```
sudo nano /etc/ssh/sshd_config.d/99-kounouz.conf
```

Contenu du fichier :

```
PermitRootLogin no  
PasswordAuthentication no  
KbdInteractiveAuthentication no
```

```
sudo systemctl restart ssh
```

Installez fail2ban (bloque les tentatives de connexion répétées) et les mises à jour de sécurité automatiques :

```
sudo apt install -y fail2ban unattended-upgrades
```

```
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

En cas de perte de votre clé SSH, hPanel propose une **console navigateur** et la réinitialisation du mot de passe root.

6.6 Fichier d'échange (conseillé sur KVM 1)

La construction du site consomme beaucoup de mémoire pendant quelques minutes. Sur 4 Go de RAM, ajoutez 2 Go d'échange :

```
sudo fallocation -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile
```

```
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

7. Faire pointer le nom de domaine

Chez votre registraire (hPanel → **Domaines** → **DNS / Serveurs de noms** si le domaine est chez Hostinger), créez ou modifiez ces enregistrements :

Type	Nom	Valeur	TTL
A	@	IP_DU_VPS	3600
A	www	IP_DU_VPS	3600

Supprimez les anciens enregistrements A ou AAAA de @ et www qui pointeraient ailleurs. La propagation prend de quelques minutes à quelques heures. Vérifiez depuis votre ordinateur :

```
nslookup VOTRE_DOMAINE
```

L'adresse affichée doit être IP_DU_VPS . **Attendez que ce soit le cas avant la section 10** (HTTPS) : Let's Encrypt doit joindre votre serveur par ce nom.

8. Installer Docker

Sur le serveur (utilisateur `deploy`), installez Docker Engine et Docker Compose depuis le dépôt officiel de Docker :

```
sudo apt install -y ca-certificates curl git
```

```
sudo install -m 0755 -d /etc/apt/keyrings && sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc && sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt update && sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

Autorisez **deploy** à utiliser Docker sans **sudo**, puis reconnectez-vous pour que ce soit pris en compte :

```
sudo usermod -aG docker deploy
```

```
exit
```

```
ssh deploy@IP_DU_VPS
```

```
docker compose version
```

Limiter la taille des journaux Docker

Sans limite, les journaux des conteneurs finissent par remplir le disque :

```
sudo nano /etc/docker/daemon.json
```

```
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "20m", "max-file": "5" }
}
```

```
sudo systemctl restart docker
```

9. Installer l'application

9.1 Récupérer le code

Le dépôt est privé : créez une **clé de déploiement** en lecture seule sur le serveur.

```
ssh-keygen -t ed25519 -C "kounouz-vps" -f ~/.ssh/kounouz_deploy -N ""
```

```
cat ~/.ssh/kounouz_deploy.pub
```

Sur GitHub : dépôt → **Settings** → **Deploy keys** → **Add deploy key**, collez la clé, **sans** cocher « Allow write access ». Puis sur le serveur :

```
printf 'Host github.com\n  IdentityFile ~/.ssh/kounouz_deploy\n  IdentitiesOnly yes\n' >> ~/.ssh/config
```

```
sudo mkdir -p /opt/kounouz && sudo chown deploy:deploy /opt/kounouz
```

```
git clone git@github.com:VOTRE_COMPTE/kounouz-alafiya.git /opt/kounouz
```

```
cd /opt/kounouz
```

9.2 Configurer l'environnement de production

```
cp .env.deploy.example .env && chmod 600 .env
```

Générez trois secrets aléatoires (notez-les, ils vont dans `.env`) :

```
for n in JWT_ACCESS_SECRET JWT_REFRESH_SECRET FILES_SIGNING_SECRET; do echo "$n=$(openssl rand -hex 48)";  
done; echo "POSTGRES_PASSWORD=$(openssl rand -hex 24)"
```

Éditez le fichier :

```
nano .env
```

Valeurs de production (les autres lignes restent telles quelles) :

```
COMPOSE_PROJECT_NAME=kounouz

POSTGRES_USER=kounouz
POSTGRES_PASSWORD=MOT_DE_PASSE_GÉNÉRÉ
POSTGRES_DB=kounouz_alafiya
POSTGRES_PORT=5433

JWT_ACCESS_SECRET=SECRET_GÉNÉRÉ_1
JWT_REFRESH_SECRET=SECRET_GÉNÉRÉ_2
FILES_SIGNING_SECRET=SECRET_GÉNÉRÉ_3

PUBLIC_APP_URL=https://VOTRE_DOMAINE
VITE_API_URL=/api
CORS_ORIGINS=https://VOTRE_DOMAINE,https://www.VOTRE_DOMAINE

KOUNOUZ_COMMISSION_RATE=0.2
THROTTLE_TTL=60000
THROTTLE_LIMIT=120

TRUST_PROXY=2
FRONTEND_PORT=127.0.0.1:8080
BACKEND_PORT=3000
```

Variable	Pourquoi cette valeur
COMPOSE_PROJECT_NAME	Nom fixe du projet Docker, donc noms de volumes stables
PUBLIC_APP_URL	Encodée dans chaque QR code imprimé. À fixer définitivement avant de générer le moindre QR : un QR imprimé avec une mauvaise adresse est inutilisable
CORS_ORIGINS	N'autorise que votre domaine à appeler l'API depuis un navigateur
TRUST_PROXY=2	Deux proxys devant l'API (Caddy puis nginx) : l'API lit ainsi l'IP réelle des visiteurs (limite de débit, anti-contrefaçon, audit)
FRONTEND_PORT=127.0.0.1:8080	Le site n'est joignable que par Caddy, qui ajoute le HTTPS

Ne changez plus `POSTGRES_PASSWORD` une fois la base créée : PostgreSQL garde le mot de passe du premier démarrage. Pour le modifier plus tard, il faut aussi le changer dans la base.

9.3 Lancer l'application

```
docker compose up -d --build
```

La première construction prend 5 à 15 minutes selon le VPS. Suivez l'état :

```
docker compose ps
```

Les trois services doivent être **running**, puis **healthy** au bout d'une minute environ. Contrôles depuis le serveur :

```
curl -s http://127.0.0.1:3000/health
```

```
curl -s http://127.0.0.1:3000/ready
```

```
curl -sI http://127.0.0.1:8080 | head -1
```

Réponses attendues : `{"status":"ok",...}`, puis `{"status":"ready","checks":{"database":"up","eventQueue":"up"},...}`, puis `HTTP/1.1 200 OK`. En cas de problème, lisez les journaux de l'API :

```
docker compose logs --tail 100 backend
```

10. Activer le HTTPS avec Caddy

Caddy obtient et renouvelle automatiquement les certificats Let's Encrypt. Installation depuis son dépôt officiel :

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https curl
```

```
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/gpg.key' | sudo gpg --dearmor -o  
/usr/share/keyrings/caddy-stable-archive-keyring.gpg
```

```
curl -1sLf 'https://dl.cloudsmith.io/public/caddy/stable/debian.deb.txt' | sudo tee  
/etc/apt/sources.list.d/caddy-stable.list
```

```
sudo apt update && sudo apt install -y caddy
```

Configuration :

```
sudo nano /etc/caddy/Caddyfile
```

Remplacez tout le contenu par :

```
{
    email VOTRE_EMAIL
}

www.VOTRE_DOMAINE {
    redir https://VOTRE_DOMAINE{uri} permanent
}

VOTRE_DOMAINE {
    encode zstd gzip
    header Strict-Transport-Security "max-age=31536000; includeSubDomains"
    reverse_proxy 127.0.0.1:8080
}
```

```
sudo caddy validate --config /etc/caddy/Caddyfile && sudo systemctl reload caddy
```

Ouvrez https://VOTRE_DOMAINE : le cadenas doit apparaître. <http://> et www. redirigent automatiquement vers https://VOTRE_DOMAINE. En cas d'échec du certificat, consultez :

```
sudo journalctl -u caddy --since "10 min ago" --no-pager
```

La cause est presque toujours un DNS pas encore propagé (section 7) ou le port 80 fermé (pare-feu hPanel ou UFW).

11. Créer le premier administrateur

La base de production démarre **vide**. **N'exécutez pas le seed de démonstration** (`prisma:seed`) : il crée des comptes aux mots de passe publiés dans la documentation, ainsi que des producteurs et des commandes fictifs.

Créez votre compte administrateur :

```
docker compose exec backend node prisma/create-admin.mjs VOTRE_EMAIL "Prénom Nom"
```

Le script affiche un **mot de passe provisoire une seule fois**. Connectez-vous sur https://VOTRE_DOMAINE/connexion puis changez-le immédiatement (**Paramètres** → **Sécurité et mot de passe**).

Mot de passe administrateur perdu :

```
docker compose exec backend node prisma/create-admin.mjs VOTRE_EMAIL --reset-password
```

Configuration initiale, dans l'ordre

1. **Catégories** (`/admin/categories`) : par exemple Miels › Miel de thym, Miel de romarin, Miel de jujubier (Sedra)...
2. **Laboratoires** (`/admin/laboratoires`) : laboratoires partenaires et leurs accréditations. Passez-les en *Actif* : seul un laboratoire actif peut recevoir des analyses.
3. **Équipe** (`/admin/utilisateurs` → **Ajouter un membre de l'équipe**) : comptes de l'équipe de vérification et des agents terrain.
4. **Producteurs** : ils s'inscrivent sur `/inscription/producteur` , ou vous les créez dans `/admin/producteurs` . Examinez leurs documents dans leur fiche.

Le détail de chaque écran est dans `GUIDE_UTILISATION.md` .

12. Vérifications après déploiement

Vérification	Comment	Résultat attendu
HTTPS	Ouvrir <code>http://VOTRE_DOMAINE</code>	Redirection vers <code>https://</code> , cadenas valide
Site	Page d'accueil	Tout s'affiche, prix en DT
API	<code>https://VOTRE_DOMAINE/api/v1/categories</code>	Liste JSON des catégories
Base de données	<code>curl -s http://127.0.0.1:3000/ready</code> sur le serveur	"database": "up" et "eventQueue": "up"
Connexion	<code>/connexion</code> avec le compte admin	Arrivée sur <code>/admin</code>
Envoi de fichier	Créer un producteur de test, déposer un PDF de 3 à 5 Mo dans ses documents	Envoi accepté (pas d'erreur 413)
QR codes	Générer un QR de test, le scanner avec un téléphone	La page <code>https://VOTRE_DOMAINE/verify/...</code> s'ouvre
IP réelle	Admin → Journal d'audit , dernière connexion	Votre IP publique, pas une adresse <code>172.x.x.x</code>
Base non exposée	Depuis votre ordinateur : <code>Test-NetConnection IP_DU_VPS -Port 5433</code> (PowerShell)	Échec de connexion (c'est voulu)
API non exposée	Idem avec <code>-Port 3000</code> et <code>-Port 8080</code>	Échec de connexion (c'est voulu)

Supprimez ensuite les données de test (producteur, QR) ou désactivez-les.

13. Sauvegardes automatiques et restauration

À sauvegarder chaque jour : **la base PostgreSQL** et **les fichiers** (`uploads` et `private-uploads`). Le fichier `.env` est à conserver à part, dans un gestionnaire de mots de passe : sans ses secrets, les sessions et les liens signés ne fonctionnent plus.

13.1 Script de sauvegarde

```
sudo mkdir -p /var/backups/kounouz && sudo chown deploy:deploy /var/backups/kounouz
```

```
nano /opt/kounouz/backup.sh
```

```
#!/usr/bin/env bash
# Sauvegarde quotidienne Kounouz Alafiya : base + fichiers téléversés.
set -euo pipefail
cd /opt/kounouz
source <(grep -E '^(POSTGRES_USER|POSTGRES_DB)=' .env)
DEST=/var/backups/kounouz
STAMP=$(date +%Y-%m-%d_%H%M)

docker compose exec -T postgres pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" --format=custom \
  > "$DEST/db_$STAMP.dump"
docker compose exec -T backend tar czf - -C /app uploads private-uploads \
  > "$DEST/fichiers_$STAMP.tar.gz"

# Conservation : 14 jours.
find "$DEST" -type f -mtime +14 -delete
echo "Sauvegarde terminée : $STAMP"
```

```
chmod +x /opt/kounouz/backup.sh && /opt/kounouz/backup.sh && ls -lh /var/backups/kounouz
```

13.2 Planifier la sauvegarde chaque nuit

```
crontab -e
```

Ajoutez la ligne (sauvegarde à 3 h du matin, heure de Tunis) :

```
0 3 * * * /opt/kounouz/backup.sh >> /var/backups/kounouz/backup.log 2>&1
```

13.3 Copie hors du serveur

Une sauvegarde qui reste sur le serveur disparaît avec lui. Rapatriez régulièrement les fichiers, par exemple chaque semaine, depuis votre ordinateur :

```
scp -r deploy@IP_DU_VPS:/var/backups/kounouz ./sauvegardes-kounouz
```

Pour une copie automatique vers un stockage externe (Google Drive, Backblaze B2, S3...), utilisez `rclone` .

13.4 Restaurer

Arrêtez l'API pendant la restauration, puis restaurez la base (remplacez la date) :

```
cd /opt/kounouz && docker compose stop backend
```

```
source <(grep -E '^(POSTGRES_USER|POSTGRES_DB)=' .env); docker compose exec -T postgres pg_restore -U "$POSTGRES_USER" -d "$POSTGRES_DB" --clean --if-exists < /var/backups/kounouz/db_AAAA-MM-JJ_HHMM.dump
```

Redémarrez l'API, puis restaurez les fichiers :

```
docker compose start backend
```

```
docker compose exec -T backend tar xzf - -C /app < /var/backups/kounouz/fichiers_AAAA-MM-JJ_HHMM.tar.gz
```

Testez une restauration au moins une fois, par exemple sur un second VPS temporaire : une sauvegarde jamais restaurée n'est pas une sauvegarde fiable.

14. Mettre à jour l'application

Sur votre ordinateur : testez, enregistrez et envoyez les modifications.

```
git add -A && git commit -m "Description de la modification" && git push
```

Sur le serveur :

1. Prenez un **instantané** dans hPanel (VPS → Instantanés), ou au minimum une sauvegarde :

```
/opt/kounouz/backup.sh
```

2. Récupérez le code et reconstruisez :

```
cd /opt/kounouz && git pull && docker compose up -d --build
```

Les nouvelles migrations sont appliquées automatiquement au démarrage de l'API. Vérifiez :

```
docker compose ps && docker compose logs --tail 50 backend
```

3. Faites le ménage des anciennes images :

```
docker image prune -f
```

Revenir en arrière

Si la nouvelle version pose problème, revenez au commit précédent :

```
cd /opt/kounouz && git log --oneline -5
```

```
git checkout IDENTIFIANT_DU_COMMIT_PRÉCÉDENT && docker compose up -d --build
```

Si une migration a modifié la base, restaurez aussi la sauvegarde prise juste avant (section 13.4) ou l'instantané hPanel. Revenez ensuite sur la branche principale avec `git checkout master` une fois le problème corrigé.

15. Surveillance et maintenance

Commandes utiles

Besoin	Commande (dans <code>/opt/kounouz</code>)
État des services	<code>docker compose ps</code>
Journaux de l'API en direct	<code>docker compose logs -f backend</code>
Journaux du site	<code>docker compose logs --tail 100 frontend</code>
Redémarrer l'API	<code>docker compose restart backend</code>
Espace disque	<code>df -h</code> et <code>docker system df</code>
Mémoire et processeur	<code>htop</code> (installez-le avec <code>sudo apt install htop</code>)
Console SQL	<code>docker compose exec postgres psql -U kounouz kounouz_alafiya</code>

Les conteneurs redémarrent seuls après une panne ou un redémarrage du serveur (`restart: unless-stopped`).

Surveillance de disponibilité

Créez une sonde gratuite sur UptimeRobot (ou Better Stack) qui vérifie toutes les 5 minutes :

- `https://VOTRE_DOMAINE/` (le site) ;
- `https://VOTRE_DOMAINE/api/v1/categories` (l'API et la base).

Vous recevrez un e-mail en cas de panne.

Accéder à la base depuis votre ordinateur

La base n'est pas exposée sur Internet. Pour l'ouvrir avec Prisma Studio, DBeaver ou pgAdmin, créez un tunnel SSH :

```
ssh -L 5433:127.0.0.1:5433 deploy@IP_DU_VPS
```

Tant que ce terminal reste ouvert, la base est accessible sur `localhost:5433` depuis votre ordinateur.

Tâches régulières

Fréquence	Tâche
Chaque semaine	Vérifier <code>backup.log</code> , rapatrier les sauvegardes, contrôler l'espace disque
Chaque mois	<code>sudo apt update && sudo apt upgrade -y</code> , puis redémarrer si demandé ; <code>docker image prune -f</code>
Chaque trimestre	Tester une restauration ; revoir les comptes actifs (Utilisateurs & rôles)

16. Avant d'ouvrir au public

Ces points ne bloquent pas l'installation technique, mais ils sont visibles des clients :

Élément	Où	Action
Numéro de téléphone provisoire +216 71 000 100 (aussi utilisé pour WhatsApp)	<code>frontend/src/lib/site-contact.ts</code>	Mettre le vrai numéro
Adresse (ville seulement)	Traductions <code>marketplace.json</code> , clé <code>footer.address</code>	Compléter si souhaité
Codes QR de démonstration proposés à l'essai	Accueil et page « Vérifier un produit »	Retirer les boutons <code>KZ-QR-2026-000001</code> / <code>000002</code>
Politique de confidentialité, conditions, retours	Pied de page (simples messages d'alerte aujourd'hui)	Rédiger de vraies pages
Note « 4,9/5 » et avis clients	Page Boutique	Retirer tant qu'il n'existe pas de vrais avis
Frais de livraison (25 DT, gratuit au-delà de 200 DT)	Panier	Confirmer les montants
Paiement	Il n'y a pas de paiement en ligne : la commande est enregistrée, le paiement se fait hors site	Définir le processus (paiement à la livraison, virement...) et l'annoncer au client
Mot de passe oublié	Il n'existe pas de réinitialisation par e-mail	L'administrateur réinitialise le mot de passe depuis Utilisateurs & rôles
Données personnelles	Les producteurs déposent leur CIN	Informez-vous de vos obligations de déclaration auprès de l'INPDP (Instance nationale de protection des données personnelles)
Comptes de démonstration	Base de production	Aucun ne doit exister (ne jamais lancer le seed)

17. Dépannage

Symptôme	Cause probable	Solution
502 Bad Gateway	L'API ou le site ne tourne pas	<code>docker compose ps</code> , puis <code>docker compose logs --tail 100 backend</code>
L'API redémarre en boucle	Migration en échec, ou secret JWT manquant dans <code>.env</code>	Lire les journaux de l'API ; corriger <code>.env</code> ; restaurer la base si une migration a échoué
Certificat HTTPS non obtenu	DNS pas encore propagé, ou port 80 fermé	<code>nslookup VOTRE_DOMAINE</code> ; vérifier les pare-feu hPanel et UFW ; <code>sudo journalctl -u caddy</code>
Erreur 413 à l'envoi d'un fichier	Fichier de plus de 10 Mo, refusé par l'API	Réduire le fichier (limite : 10 Mo pour les documents, 5 Mo pour les photos)
Erreurs 429 « Too Many Requests » pour tout le monde	<code>TRUST_PROXY</code> absent ou faux	<code>TRUST_PROXY=2</code> dans <code>.env</code> , puis <code>docker compose up -d</code>
Les QR codes pointent vers <code>localhost</code>	<code>PUBLIC_APP_URL</code> était faux au moment de la génération	Corriger <code>.env</code> , redémarrer, puis régénérer les QR non encore imprimés
La construction s'arrête (« Killed », mémoire)	RAM insuffisante pendant la construction	Ajouter le fichier d'échange (section 6.6)
Disque plein	Images et journaux Docker	<code>docker image prune -f</code> , <code>docker builder prune -f</code> , vérifier la rotation des journaux (section 8)
Impossible de se connecter en SSH	Clé perdue ou erreur dans <code>sshd_config</code>	Console navigateur de hPanel, puis corriger <code>/etc/ssh/sshd_config.d/99-kounouz.conf</code>